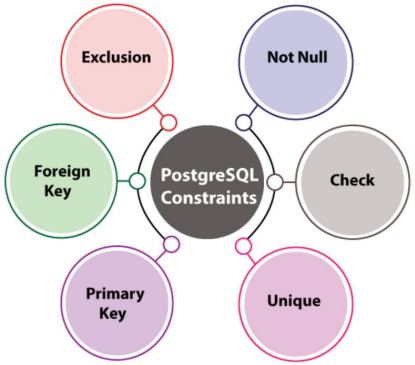
Внедрение ограничений целостности данных

Назначение ограничений целостности

- **Ограничения** – механизм Database Engine для автоматического обеспечения целостности данных в базе данных
- Ограничения обеспечивают:
  - Сущностную целостность
  - Доменную целостность
  - Ссылочную целостность

Типы ограничений целостности данных

- Разрешение значений NULL
- Определения DEFAULT значений
- Ограничения CHECK
- Ограничения UNIQUE
- Ограничения PRIMARY KEY
- Ограничения FOREIGN KEY
- Ограничение EXCLUSION



Ограничение NOT NULL

- Ограничение уровня столбца, определяемое при создании или модификации таблицы
- NOT NULL указывает, что в столбце недопустимы значения NULL

```
CREATE TABLE  "HumanResources"."Employee" (
        EmployeeID   int   IDENTITY(1,1) NOT NULL,
        NationalIDNumber   varchar(15)   NOT NULL,
        gender char(3) NULL,
        …
 );
```

```
ALTER TABLE employee
ALTER COLUMN gender SET NOT NULL;
```

Определение DEFAULT значений

- Ограничение уровня столбца, определяемое при создании или модификации таблицы
- Определяет значение по умолчанию, которым Database Engine будет заполнять столбец, если при вставке строки для этого столбца значение не указано

```
CREATE TABLE tbl_location(
     Availability   decimal(8, 2) NOT NULL DEFAULT 0.00,
     ModifiedDate  date NOT NULL
                       CONSTRAINT "DF_Location_ModifiedDate" DEFAULT now()
);
```

Изменение и удаление ограничений DEFAULT

```
ALTER TABLE <table_name>
ALTER COLUMN <column_name>
[SET DEFAULT <value> | DROP DEFAULT];
```

```
CREATE TABLE test_default(
       id serial,
       fone1 char(14),
       fone2 char(14)
);
```

Название	#	Тип данных	Not Null	По умолчанию
id	1	serial4	[v]	nextval('test_default_id_seq'::regclass)
fone1	2	bpchar(14)	[ ]	
fone2	3	bpchar(14)	[ ]	

```
ALTER TABLE test_default
ALTER COLUMN id DROP DEFAULT;
```

Название	#	Тип данных	Not Null	По умолчанию
id	1	int4	[v]	
fone1	2	bpchar(14)	[ ]	
fone2	3	bpchar(14)	[ ]	

Ограничение CHECK

- Ограничения CHECK может быть определено на уровне столбца или таблицы при создании или модификации таблицы
  - Для каждого столбца можно указать несколько *непротиворечивых* ограничений CHECK
- Ограничение CHECK — позволяет указать логическое условие для одного или нескольких столбцов, которое должно быть выполнено перед вставкой или обновлением значений
  - Использует логическое выражение для оценки значений перед их вставкой или обновлением в столбце
  - Если значения проходят проверку, PostgreSQL вставит или обновит эти значения в столбце
  - В противном случае PostgreSQL отклонит изменения и выдаст ошибку нарушения ограничения

```
Имя_столбца тип_данных [CONSTRAINT имя_ограничения] CHECK(условие_на значение)
```

- Если при создании ограничения не указать его имя PostgreSQL задаст его сам

```
{имя_таблицы}_{имя_столбца}_check
```

Создание ограничения CHECK

Имя ограничения

Тип ограничения

Условие на значения

```
CREATE TABLE test_check(
id serial,
fone1 char(14) CONSTRAINT ch_fn1 CHECK (fone1 ~ '8\([0-9]{3}\)[0-9]{3}-[0-9]{4}'),
fone2 char(14)
);
```

Ограничение уровня столбца

```
CREATE TABLE prices_list (
    id serial PRIMARY KEY,
    product_id int NOT NULL,
    price numeric NOT NULL,
    discount numeric NOT NULL,
    valid_from date NOT NULL,
    valid_to date NOT NULL,
    CONSTRAINT price_discount_check
    CHECK (price > 0 AND discount >= 0 AND price > discount)
);
```

Ограничение уровня таблицы

Добавление и удаление ограничения CHECK

Отключить проверку существующих данных

```
ALTER TABLE test_check
ADD CONSTRAINT ch_fn2 CHECK
                    (fone2 ~ '8\([0-9]{3}\)[0-9]{3}-[0-9]{4}') NOT VALID ;

INSERT INTO test_check(fone1,fone2)
VALUES ('8(921)883-9613','8(921)777-9557');
```

	id	fone1	fone2
1	1	8(921)143-9613	8 921 733 9553
2	2	8(921)883-9613	8(921)777-9557

```
ALTER TABLE test_check DROP CONSTRAINT ch_fn2 ;
```

Ограничение UNIQUE

- Может быть задано на уровне столбца или таблицы при создании или модификации таблицы
- Обеспечивает уникальность записей в таблице (альтернативный ключ)
  - При вставке новой строки, ограничение UNIQUE проверяет, является ли значение уникальным
  - Если значение не является уникальным (значение уже существует в столбце таблицы) - изменение отклоняется и выдается ошибка
  - Тот же процесс выполняется для обновления существующих данных
- Допускает наличие NULL-значений в столбце, при условии что не установлено ограничение NOT NULL
- Автоматически создается связанный с ограничением уникальный индекс

```
[CONSTRAINT constraint_name] UNIQUE (column_1, column_2,...)
```

- Если при создании ограничения не указать его имя PostgreSQL задаст его сам

```
{имя_таблицы}_{имя_столбца1_имя_столбца2…}_key
```

Определение ограничения UNIQUE

Ограничение уровня столбца

```
CREATE TABLE "Sales"."Customers"(
    custid serial NOT NULL,
    companyname varchar(40),
    email varchar(40) NULL UNIQUE,…
);
```

Ограничение уровня таблицы

```
CREATE TABLE "Sales"."Customers"(
   custid  serial NOT NULL,
   companyname varchar(40),
   email varchar(40) NULL,   …
   CONSTRAINT AK_email UNIQUE(email)
);
```

Ограничение уровня таблицы

```
CREATE TABLE table (
    c1 data_type,
    c2 data_type,
    c3 data_type,
CONSTRAINT AK_po_item UNIQUE (c2, c3)
);
```

Добавление и удаление ограничения UNIQUE

- При добавлении ограничения UNIQUE к существующему столбцу таблицы, в котором есть неуникальные значения – будет выдано сообщение об ошибке

```
ALTER TABLE "Sales"."Customers"
ADD CONSTRAINT Unique_Name UNIQUE (companyname);
```

- При удалении ограничения UNIQUE автоматически удаляется связанный с ограничением уникальный индекс

```
ALTER TABLE "Sales"."Customers"
DROP CONSTRAINT Unique_Name;
```

Ограничение PRIMARY KEY

- Может быть задано на уровне столбца или таблицы при создании или модификации таблицы
- Используется для создания первичного ключа таблицы
- В таблице может быть **только одно** ограничение PRIMARY KEY
- Столбец PRIMARY KEY **не может** содержать значения **NULL**
  - Технически, ограничение PRIMARY KEY представляет собой комбинацию ограничения NOT NULL и ограничения UNIQUE
- Автоматически создается связанный с ограничением уникальный индекс сбалансированного дерева

```
[CONSTRAINT constraint_name] PRIMARY KEY (column_1, column_2,...)
```

- Если при создании ограничения не указать его имя PostgreSQL задаст его сам

```
{имя_таблицы}_pkey
```

Определение ограничения Primary Key

Ограничение уровня столбца

```
CREATE TABLE "Sales"."Customers"(
     custid serial NOT NULL PRIMARY KEY,
     companyname varchar(40), …
);
```

Ограничение уровня таблицы

```
CREATE TABLE "Sales"."Customers"(
   custid  serial NOT NULL,
   companyname varchar(40), …
   CONSTRAINT PK_CustomerID PRIMARY KEY(custid)
);
```

Ограничение уровня таблицы

```
CREATE TABLE po_items (
   po_no integer,
   item_no integer,
   product_no integer,
   qty integer,
   net_price numeric,
   CONSTRAINT PK_po_item PRIMARY KEY (po_no, item_no)
);
```

Добавление и удаление ограничения Primary Key

- При добавлении ограничения Primary Key к существующему столбцу таблицы, в котором есть неуникальные значения – будет выдано сообщение об ошибке

```
ALTER TABLE po_items ADD PRIMARY KEY (po_no, item_no);
```

```
ALTER TABLE vendors ADD COLUMN ID SERIAL PRIMARY KEY;
```

- При попытке удаления первичного ключа, на который ссылается внешний ключ в связной таблице, будет выдано сообщение об ошибке
- Для удаления первичного ключа и связанного с ним внешнего ключа необходимо использовать указание **CASCADE**
- При удалении ограничения Primary Key автоматически удаляется связанный с ограничением уникальный индекс

```
ALTER TABLE po_items DROP CONSTRAINT po_items_pkey CASCADE ;
```

Ограничение FOREIGN KEY

- Внешний ключ — это столбец или группа столбцов в таблице, которые ссылаются на существующий **первичный** или **альтернативный ключ** другой таблицы
  - Таблица, содержащая внешний ключ, называется ссылочной таблицей или дочерней таблицей
  - Таблица, на которую ссылается внешний ключ, называется родительской таблицей
- Таблица может иметь несколько внешних ключей в зависимости от ее отношений с другими таблицами
- В PostgreSQL внешний ключ определяется с помощью ограничения **FOREIGN KEY**
  - Ограничение FOREIGN KEY помогает поддерживать ссылочную целостность данных между дочерней и родительской таблицами
  - Ограничение внешнего ключа указывает, что значения в столбце или группе столбцов дочерней таблицы равны значениям в столбце или группе столбцов родительской таблицы
- У пользователя должно быть разрешение REFERENCES для таблицы, на которую указывают ссылки
- Индекс не создается автоматически, поэтому его необходимо создать самостоятельно

Синтаксис

```
[CONSTRAINT имя_ограничения] FOREIGN KEY(список_столбцов)
REFERENCES имя_таблицы_родителя (список_столбцов)
      [ON DELETE тип_обработки]
      [ON UPDATE тип_обработки]
```

```
ALTER TABLE "Production"."Products"
      ADD CONSTRAINT "FK_Category" FOREIGN KEY (categoryid)
      REFERENCES "Production"."Categories"(categoryid)
      ON DELETE NO ACTION
      ON UPDATE CASCADE;
```

Определение ограничения Foreign Key

Ограничение уровня столбца

```
CREATE TABLE ord(
       ordid serial PRIMARY KEY,
       idcl int REFERENCES cl(id),
       dt date);
```

Ограничение уровня таблицы

```
CREATE TABLE ord(
       ordid serial PRIMARY KEY,
       idcl int,
       CONSTRAINT fk_idcl FORIGN KEY(idcl)
       REFERENCES cl(id),
       dt date);
```

Ограничение уровня таблицы

```
CREATE TABLE ord (
       ordid serial PRIMARY KEY,
       passport_ser char(3),
       passport_num char(6),
       CONSTRAINT fk_idcl FORIGN KEY (passport_ser, passport_num)
       REFERENCES cl(passport_ser, passport_num),
       dt date);
```

Ссылочная целостность

CategoryID	ParentProductCategoryID	Name	…
1	NULL	Bikes	…
2	NULL	Components	…
…	…	…	…
6	1	Road bikes	…
8	2	Handlebars	

Первичный ключ

Ссылка

Внешний ключ

ProductID	Name	ProductNumber	CategoryID	…
749	Road-150 Red, 62	BK-R93R-62	6	…
750	Road-150 Red, 44	BK-R93R-44	6	…
…	…	…	…	…
808	LL Mountain Handlebars	HB-M243	8	…

Правила удаления и обновления

```
ALTER TABLE имя_таблицы
ADD CONSTRAINT
имя_ограничения FOREIGN KEY(список_столбцов)
REFERENCES имя_таблицы_родителя (список_столбцов)
 [ON DELETE тип_обработки]
 [ON UPDATE тип_обработки]
```

тип_обработки = SET NULL | SET DEFAULT | RESTRICT | **NO ACTION** | CASCADE

Правила удаления и обновления

```
UPDATE category
SET categoryid = 11
WHERE categoryid = 1;
```

categoryid	categoryname
1	Beverages
2	Condiments
3	Confections
4	Dairy Products
5	Grains/Cereals
6	Meat/Poultry
7	Produce
8	Seafood

```
DELETE category
WHERE categoryid = 7;
```

ON UPDATE

ON DELETE

Тип обработки

NO ACTION
SET NULL
SET DEFAULT
RESTRICT
CASCADE

productid	productname	categoryid
1	Product HHYDP	1
2	Product RECZE	1
3	Product IMEHJ	2
4	Product KSBRM	2
5	Product EPEIM	2
6	Product VAIIV	2
7	Product HMLNI	7
8	Product WVJFP	2
9	Product AOZBW	6
10	Product YHXGE	8

Тип обработки

- **NO ACTION** – если при проверке ограничения обнаруживаются зависимые записи, возникает ошибка (это поведение по умолчанию)
  - NO ACTION позволяет отложить проверку в процессе транзакции
- **RESTRICT** – предотвращает удаление связанной записи
  - Действие RESTRICT аналогично действию NO ACTION
  - Разница возникает только при определении ограничения внешнего ключа как ***DEFERRABLE*** с режимом ***INITIALLY DEFERRED*** или ***INITIALLY IMMEDIATE***
- **CASCADE** – автоматическое удаление зависимых записей при удалении связанных записей
- **SET NULL** – замена значений внешнего ключа на **NULL** в зависимых записях при удалении связанных записей
- **SET DEFAULT** – замена значений внешнего ключа на значение по умолчанию в зависимых записях при удалении связанных записей
  - Необходимо определение ограничения ***DEFAULT*** для внешнего ключа
  - Необходимо наличие записи в главной таблице с соответствующим значением первичного ключа

Каскадные обновления

categoryid	categoryname
1	Beverages
12	Condiments
3	Confections
4	Dairy Products
5	Grains/Cereals
6	Meat/Poultry
7	Produce
8	Seafood

CASCADE

productid	productname	categoryid
1	Product HHYDP	1
2	Product RECZE	1
3	Product IMEHJ	12
4	Product KSBRM	12
5	Product EPEIM	12
6	Product VAIIV	12
7	Product HMLNI	7
8	Product WVJFP	12
9	Product AOZBW	6
10	Product YHXGE	8

```
CREATE TABLE product(
        productid serial PRIMARY KEY,
        Productname varchar(15),
        categoryid int,
        CONSTRAINT fk_catid FORIGN KEY(categoryid)
        REFERENCES category(categoryid)
        ON UPDATE CASCADE
        ON DELETE NO ACTION);
```

Каскадные удаления

categoryid	categoryname
1	Beverages
2	Condiments
3	Confections
4	Dairy Products
5	Grains/Cereals
6	Meat/Poultry
7	Produce
8	Seafood

CASCADE

productid	productname	categoryid
1	Product HHYDP	1
2	Product RECZE	1
3	Product IMEHJ	2
4	Product KSBRM	2
5	Product EPEIM	2
6	Product VAIIV	2
7	Product HMLNI	7
8	Product WVJFP	2
9	Product AOZBW	6
10	Product YHXGE	8

CASCADE

orderid	productid	unitprice
10248	1	14
10248	2	9,8
10248	4	34,8
10249	7	18,6
10249	3	42,4
10250	5	7,7
10250	1	42,4
10250	6	16,8
10251	6	16,8
10251	8	15,6
10251	7	16,8
10252	1	64,8
10252	3	2

Получение информации о существующих ограничениях

- Тип ограничения в системном каталоге указывается одним из символов:

```
SELECT conname, contype
FROM pg_catalog.pg_constraint;
```

- “p” - PRIMARY KEY
- “f” – FOREIGN KEY
- “c” - CHECK
- “u” - UNIQUE

```
SELECT table_schema, table_name, column_name,
       conname, contype, condeferrable
FROM   pg_catalog.pg_constraint
       JOIN
       information_schema.constraint_column_usage
       ON constraint_name = conname
```